

Worksheet No. - 04

Student Name: Sukhleen Kaur

Branch: MCA

Semester: 2nd

Subject Name: Artificial Intelligence Tools

UID: 25MCA20031

Section/Group: 25MCA-6B

Date of Performance: 03.03.2026

Subject Code: 25CAP-657

Aim/Overview of the practical: Implement A* Search Algorithm.

Objective: To develop an efficient pathfinding algorithm that finds the shortest path between a start node and a goal node using both actual cost and heuristic estimation.

Procedure/Algorithm/Code

Algorithm

- 1) Start
- 2) Create an empty open list (priority queue).
- 3) Add the start node to the open list.
- 4) Create an empty closed list.
- 5) Repeat until open list is empty:
 - a) Remove the node with the lowest $f(n)$ value.
 - b) If it is the goal node, stop and reconstruct the path.
 - c) Otherwise, add it to the closed list.
 - d) For each neighbour:
 - a. Calculate
$$f(n) = g(n) + h(n)$$
 - b. If neighbour is not in closed list:
 - i. Add it to open list.
- 6) If goal is not found, return No Path Found.
- 7) Stop

Code:

```
from queue import PriorityQueue

class Node:

    def __init__(self, position, parent=None, g=0, h=0):

        self.position = position

        self.parent = parent

        self.g = g

        self.h = h

        self.f = self.g + self.h

    def __lt__(self, other):

        return self.f < other.f

def heuristic(a, b):

    """Manhattan Distance"""

    return abs(a[0] - b[0]) + abs(a[1] - b[1])

def a_star(grid, start, goal):

    open_list = PriorityQueue()

    open_list.put(Node(start, None, 0, heuristic(start, goal)))

    closed_list = set()

    while not open_list.empty():

        current_node = open_list.get()
```

```
current_position = current_node.position
```

```
if current_position == goal:
```

```
    path = []
```

```
    while current_node:
```

```
        path.append(current_node.position)
```

```
        current_node = current_node.parent
```

```
    return path[::-1]
```

```
closed_list.add(current_position)
```

```
for dx, dy in [(-1, 0), (1, 0), (0, -1), (0, 1)]:
```

```
    neighbour = (current_position[0] + dx,
```

```
                current_position[1] + dy)
```

```
    if (0 <= neighbour[0] < len(grid) and
```

```
        0 <= neighbour[1] < len(grid[0]) and
```

```
        grid[neighbour[0]][neighbour[1]] != 'X' and
```

```
        neighbour not in closed_list):
```

```
        g = current_node.g + 1
```

```
        h = heuristic(neighbour, goal)
```

```
        neighbour_node = Node(neighbour, current_node, g, h)
```

```
    if neighbour not in [node.position for node in open_list.queue]:
```

```
        open_list.put(neighbour_node)

    return None

if __name__ == "__main__":
    grid = [
        ['S', '.', '.', '.', '.', '.', 'X', '.', '.', '.'],
        ['.', 'X', 'X', '.', 'X', '.', 'X', '.', 'X', '.'],
        ['.', '.', '.', '.', 'X', '.', '.', '.', 'X', '.'],
        ['X', '.', 'X', '.', '.', '.', 'X', '.', '.', '.'],
        ['.', '.', 'X', 'X', 'X', '.', '.', 'X', 'G', '.'],
        ['.', '.', '.', '.', '.', '.', '.', '.', '.', '.']
    ]

    start = (0, 0)

    goal = (4, 8)

    path = a_star(grid, start, goal)

    if path:
        print("Path found from start to goal:")

        print(path)
    else:
        print("Path not found!!!")
```

Output

Path found from start to goal:

```
[(0, 0), (0, 1), (0, 2), (0, 3), (0, 4), (0, 5), (1, 5), (2, 5), (2, 6), (2, 7), (3, 7), (3, 8), (4, 8)]
```

Learning outcomes (What I have learnt):

- 1) Understood the concept of A* search algorithm.
- 2) Apply the formula $f(n) = g(n) + h(n)$.
- 3) Used heuristic functions for pathfinding.
- 4) Implemented A* using Priority Queue in Python.